Prototype

Write-up

The states, edge cases, data model and student messages behind this prototype.

States

State	Enters when	Leaves to
Not eligible	Skills still missing	Eligible: last skill cleared
Eligible	All skills cleared, or no-show, or cool-off over	Booked: student books a slot
Booked	Slot reserved in one atomic step	Eligible: cancel or no-show. Passed, Failed or Retake: interviewer records
Passed	Interviewer records pass, badge issued	Nullified: malpractice flag
Failed, Retake later	Interviewer records, `retry_after` set (default 7 days)	Eligible once `retry_after` passes
Nullified	Malpractice flag on a passed badge	Terminal (team can reinstate by hand)

Edge cases and assumptions

- Two students, one slot: the second sees "slot just taken" and the list refreshes.
- One active booking per student per badge. Cancel first to rebook.
- A recorded slot never reopens, and a slot whose hour has passed cannot be booked.
- Interviewer closes a booked hour: blocked. The team reassigns or cancels and notifies the student.
- No-show: slot freed, student returns to Eligible with no penalty (assumption).
- Student cancels: allowed up to 24 hours before the slot, then the button is replaced by a note.
- Fail and retake later both lock rebooking until `retry_after`. Retake means close, one more look.
- Malpractice flag after a pass: badge nullified, student sees the reason. A flag before the interview blocks booking.
- All times are IST (assumption). Open questions: maximum retakes, interviewer conflicts of interest, who handles appeals.

Data model

```
students(id, name, skills_cleared, eligible_at)
interviewers(id, name)
slots(id, interviewer_id, starts_at, ends_at, status[open|booked|done],
      UNIQUE(interviewer_id, starts_at))
bookings(id, slot_id, student_id, badge_id, status[active|cancelled|no_show|done], created_at,
         UNIQUE(slot_id) WHERE status='active',
         UNIQUE(student_id, badge_id) WHERE status='active')
outcomes(id, booking_id UNIQUE, result[pass|fail|retake], retry_after, notes, recorded_by, recorded_at)
badges(student_id, badge_id, status[earned|nullified], nullified_reason, PRIMARY KEY(student_id, badge_id))
```

No double booking. A partial unique index on `bookings(slot_id) WHERE status='active'`. Two simultaneous inserts: one wins, the other gets a constraint error and the UI shows "slot just taken". No app-level locking needed. The prototype mimics this with a check-and-set on the slot.

When the data is big

- **Never draw every row.** Roster and Today use a windowed list: about 20 rows exist in the page at any time, however many students there are. In production, page from the server with keyset pagination (`WHERE (starts_at, id) > (?, ?) LIMIT 50`).
- **Never scan.** Lookups go through Maps built once per change (by id, by date, by student). In the database these are indexes on `slots(starts_at)`, `bookings(student_id)` and `bookings(slot_id)`.
- **Load only what is on screen.** Booking reads one day at a time, and shows 20 interviewers with a Show more button.
- **Cheap input.** Search is debounced and only repaints the list. Saves are debounced into one write, and a full browser store is caught and reported.
- **Counts without counting.** Roster totals come from one pass here. In production keep counters updated inside the booking transaction, or a materialized view.
- Try it: Roster, then Load 10,000 demo students.

Student messages

Shown exactly like this on the student Home screen.

Booking confirmed

Your interview is booked for Fri, 14 Nov at 10:00 AM with Priya.
You can cancel up to 24 hours before.

Passed

You passed. Your Full-Stack Track Badge is now on your profile.

Retake later

You're almost there. Your interviewer wants a second look.
You can rebook from Fri, 21 Nov.